Analise

September 14, 2022

[ ]:
```
!pip uninstall matplotlib
!pip install matplotlib==3.1.3
```

[ ]:
```
pip install pmdarima
```

[ ]:
```
!pip install --upgrade xlrd
```

[ ]:
```
import pmdarima as pm
import pandas as pd, numpy as np, seaborn as sns, matplotlib.pylab as plt
from datetime import datetime
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose
```

[ ]:
```
from google.colab import auth
from google.colab import drive
auth.authenticate_user()
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

[ ]:
```
myfile = '/content/drive/Shareddrives/Modelagem Analítica/
  SERIE_MENSAL_SOBRADINHO_NOVO.xlsx'
```

[ ]:
```
database = pd.read_excel(myfile)
database
```

[ ]:

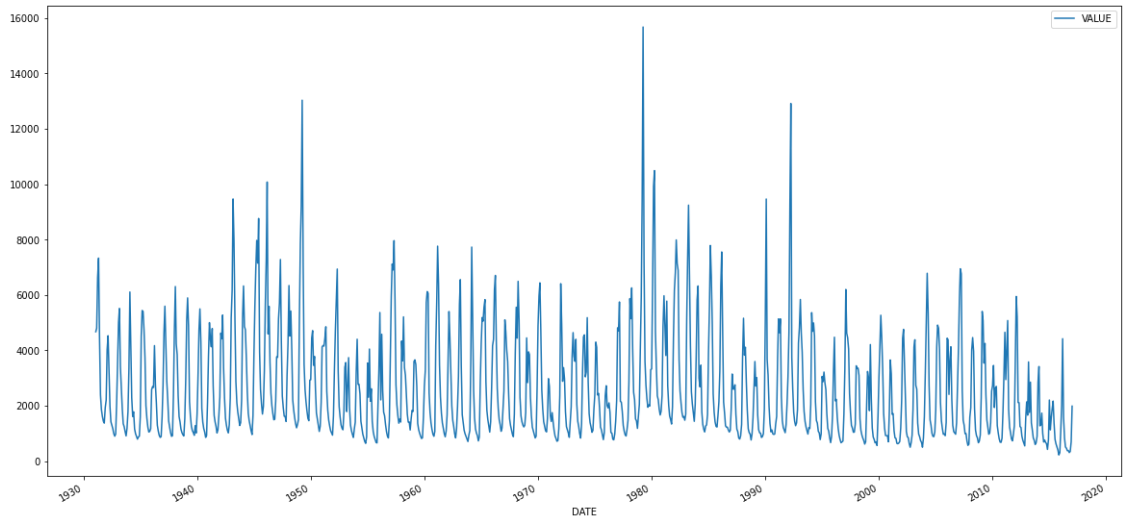
	DATE	VALUE
0	1931-01-30	4676.0
1	1931-02-28	4805.0
2	1931-03-30	6630.0
3	1931-04-30	7333.0
4	1931-05-30	4467.0
…	…	…
1027	2016-08-30	386.0
1028	2016-09-30	313.0
1029	2016-10-30	354.0
1030	2016-11-30	704.0

```
1031 2016-12-30  1989.0

[1032 rows x 2 columns]
```

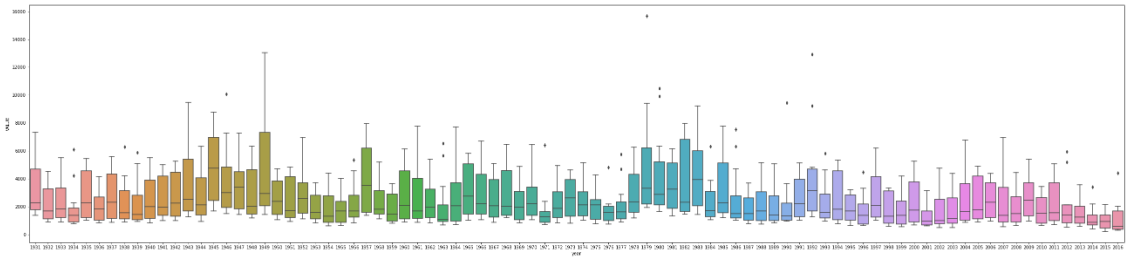
[ ]:
```
format = database
format['DATE'] = pd.to_datetime(format['DATE'])
format.set_index('DATE', inplace=True)
format.plot(figsize=(20, 10))
```

[ ]:
```
<matplotlib.axes._subplots.AxesSubplot at 0x7f9d76269090>
```

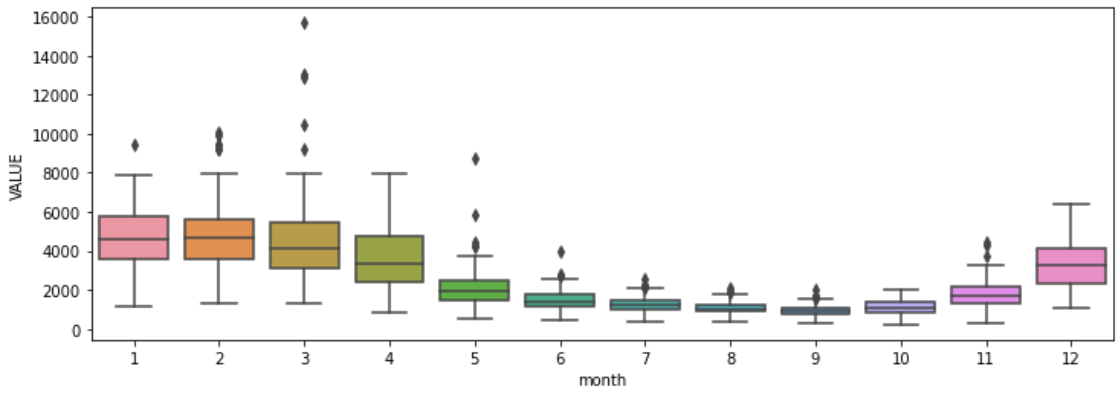


[ ]:
```
format.reset_index(inplace=True)
```

[ ]:
```
format['year'] = format['DATE'].dt.year
fig, ax = plt.subplots()
fig.set_size_inches((45,10))
sns.boxplot(x='year',y='VALUE',data=format,ax=ax)
plt.show()
```



[ ]:
```
format['month'] = format['DATE'].dt.month
fig, ax = plt.subplots()
fig.set_size_inches((12,4))
sns.boxplot(x='month',y='VALUE',data=format,ax=ax)
plt.show()
```

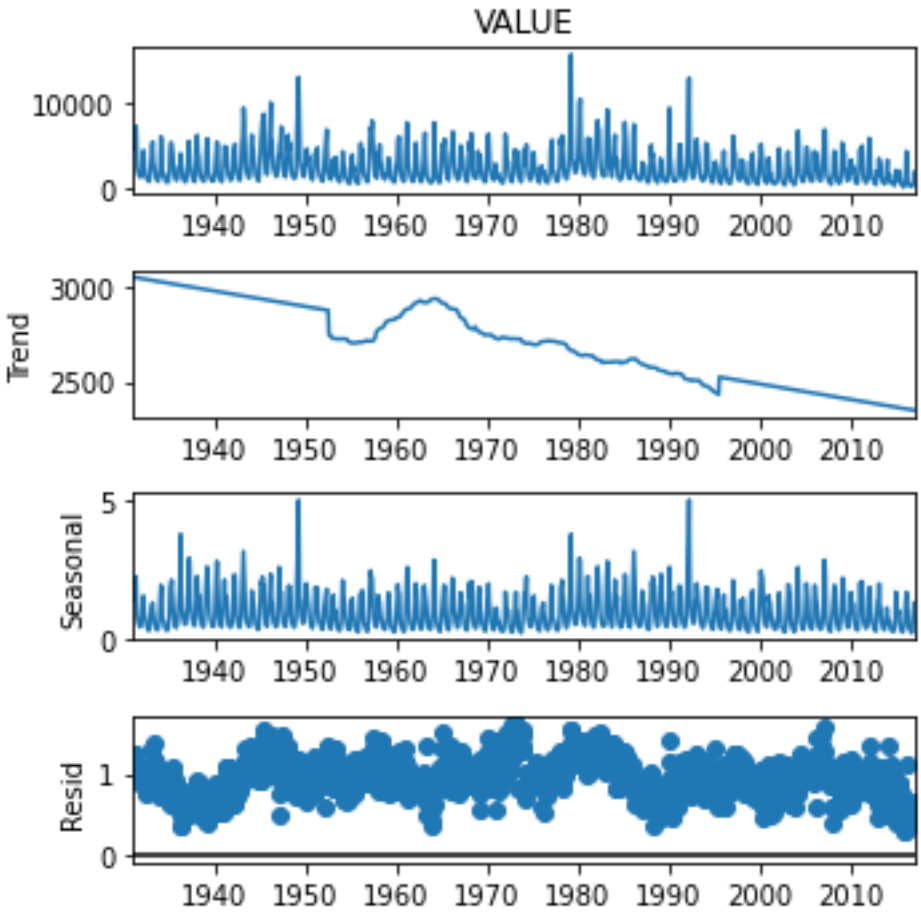


[ ]:
```
avrg = format['VALUE'].to_list()
result = adfuller(avrg)
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
  print('\t%s: %.3f' % (key, value))
```

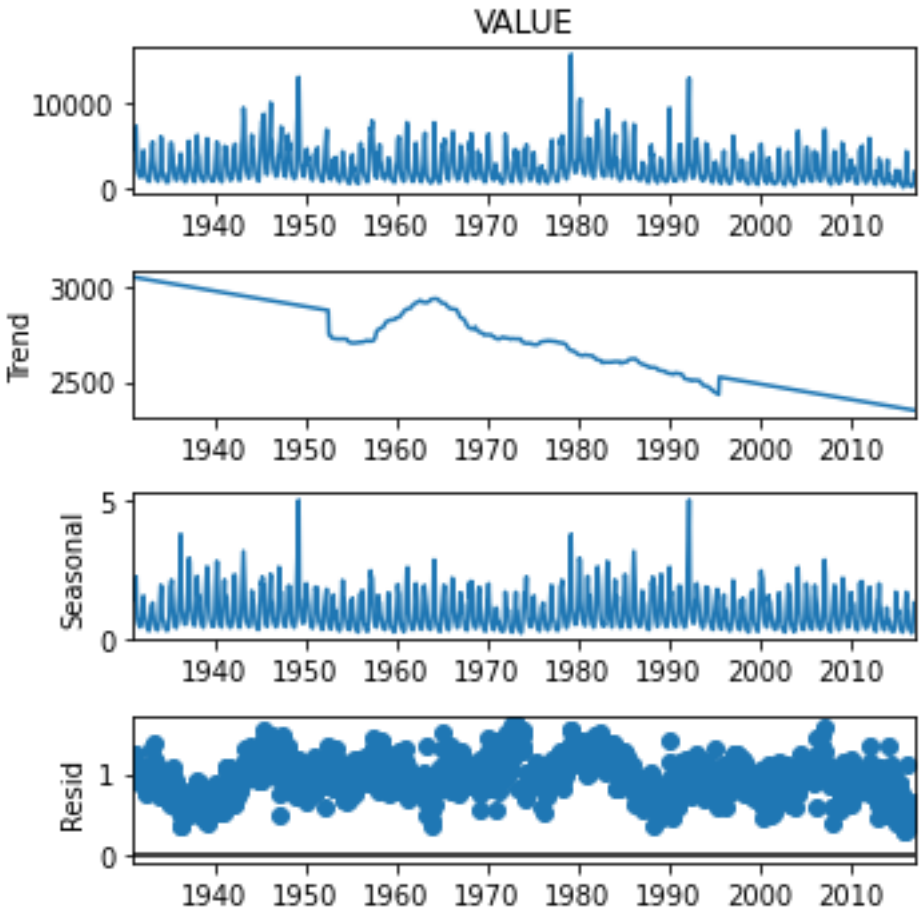
```
ADF Statistic: -3.134847
p-value: 0.024080
Critical Values:
        1%: -3.437
        5%: -2.864
        10%: -2.568
```

[ ]:
```
format.set_index(['DATE'], inplace=True)
```

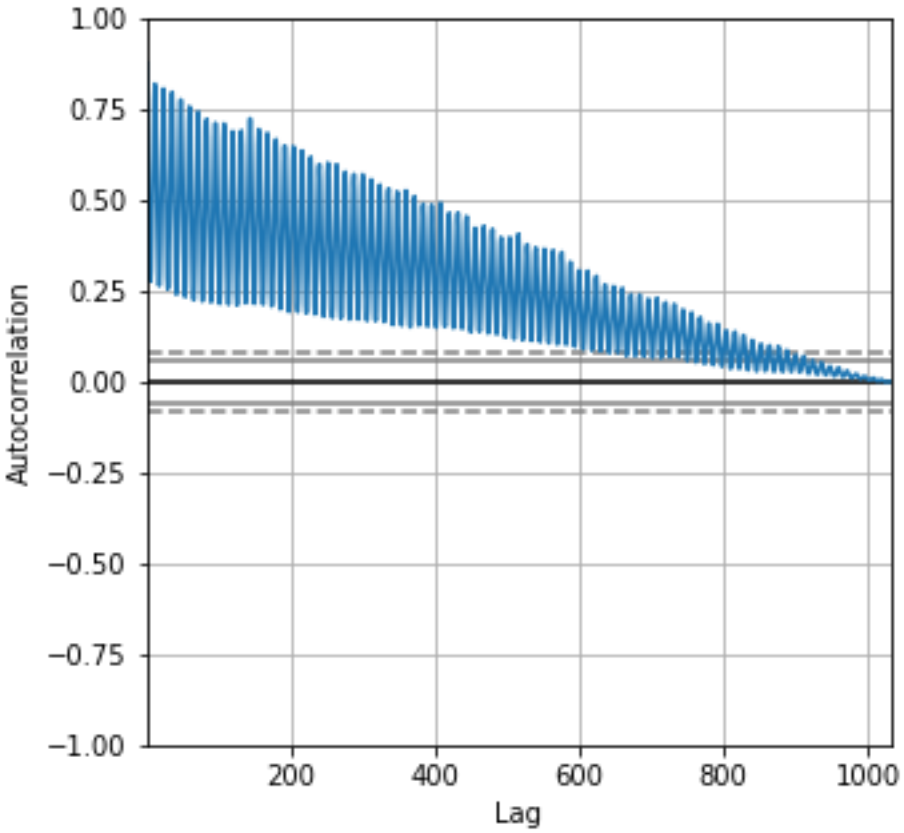
[ ]:
```
result_add = seasonal_decompose(x=format['VALUE'], model='multiplicative',␣
   extrapolate_trend='freq', period = int(len(format)/2))
plt.rcParams.update({'figure.figsize': (5,5)})
result_add.plot()
plt.show()
```



[ ]:
```
esult_add = seasonal_decompose(x=format['VALUE'], model='additive',␣
   extrapolate_trend='freq', period = int(len(format)/2))
plt.rcParams.update({'figure.figsize': (5,5)})
result_add.plot()
plt.show()
```



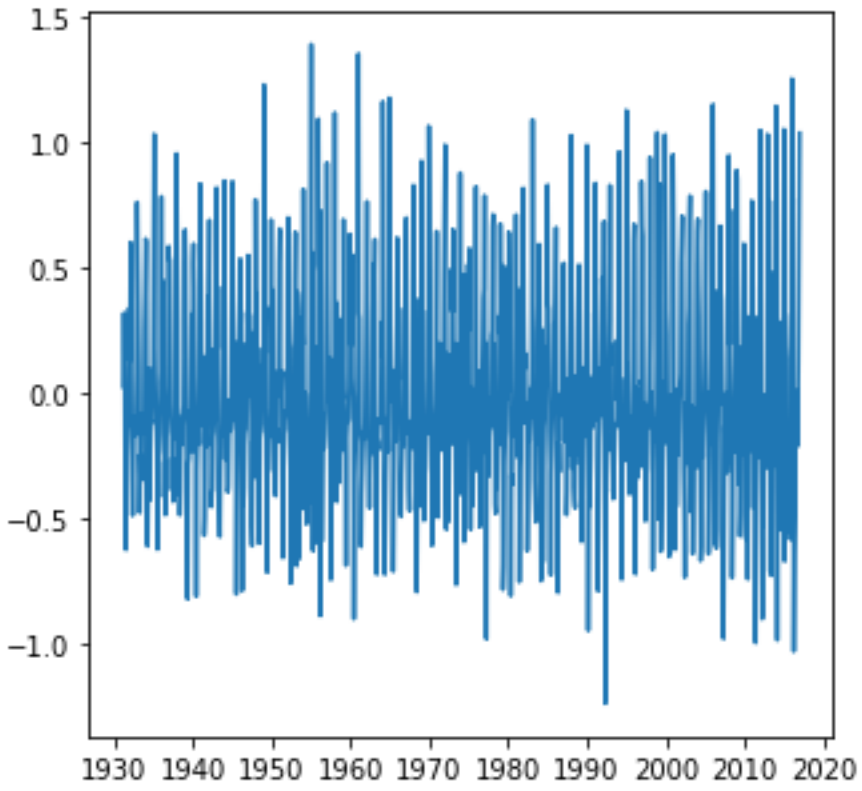
[ ]:
```
# Creating Autocorrelation plot
x = pd.plotting.autocorrelation_plot(format)
# plotting the Curve
x.plot()
# Display
plt.show()
```



[ ]:
```
from statsmodels.tsa.stattools import acf, pacf
import numpy as np
```

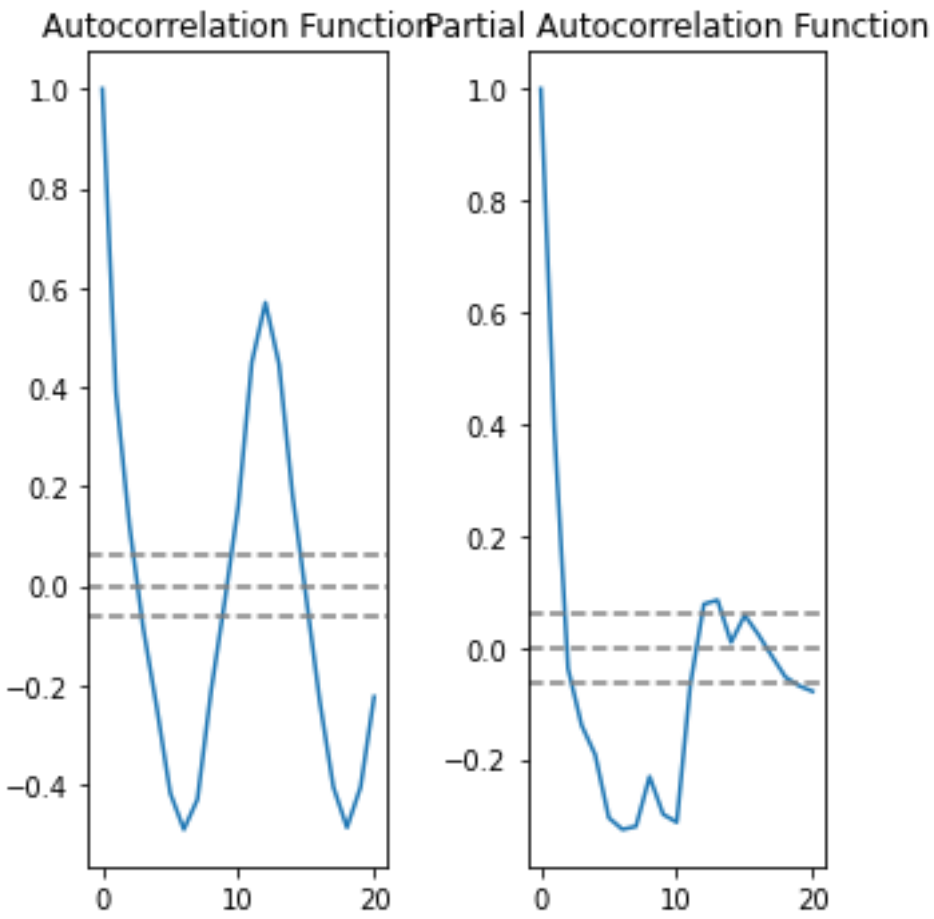
[ ]:
```
ts_log = np.log(format['VALUE'])
ts_log_diff = ts_log - ts_log.shift()
ts_log_diff.dropna(inplace=True)
plt.plot(ts_log_diff)
```

[ ]:
```
[<matplotlib.lines.Line2D at 0x7f9d706a7e50>]
```



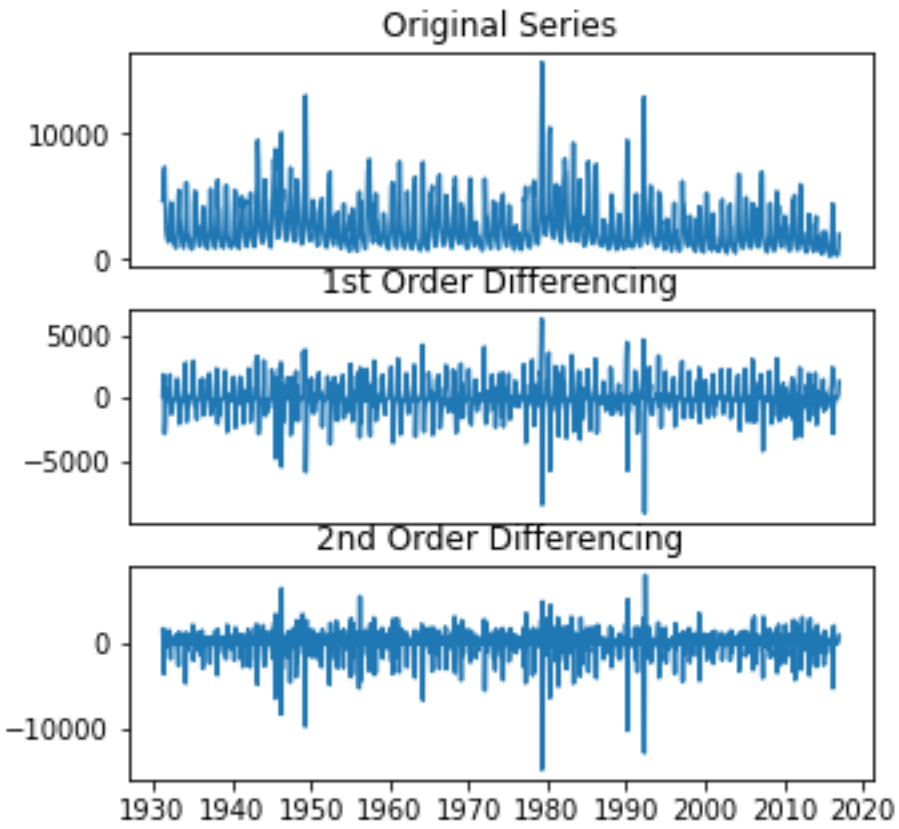
[ ]:
```
#The ACF and PACF functions are userful for setting the parameters ARIMA

lag_acf = acf(ts_log_diff, nlags=20)
lag_pacf = pacf(ts_log_diff, nlags=20)
#Plot ACF:
plt.subplot(121)
plt.plot(lag_acf)
plt.axhline(y=0,linestyle='--',color='gray')
plt.axhline(y=-1.96/np.sqrt(len(ts_log_diff)),linestyle='--',color='gray')
plt.axhline(y=1.96/np.sqrt(len(ts_log_diff)),linestyle='--',color='gray')
plt.title('Autocorrelation Function')
#Plot PACF:
plt.subplot(122)
plt.plot(lag_pacf)
plt.axhline(y=0,linestyle='--',color='gray')
plt.axhline(y=-1.96/np.sqrt(len(ts_log_diff)),linestyle='--',color='gray')
plt.axhline(y=1.96/np.sqrt(len(ts_log_diff)),linestyle='--',color='gray')
plt.title('Partial Autocorrelation Function')
plt.tight_layout()
```



[ ]:
```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import adfuller
```

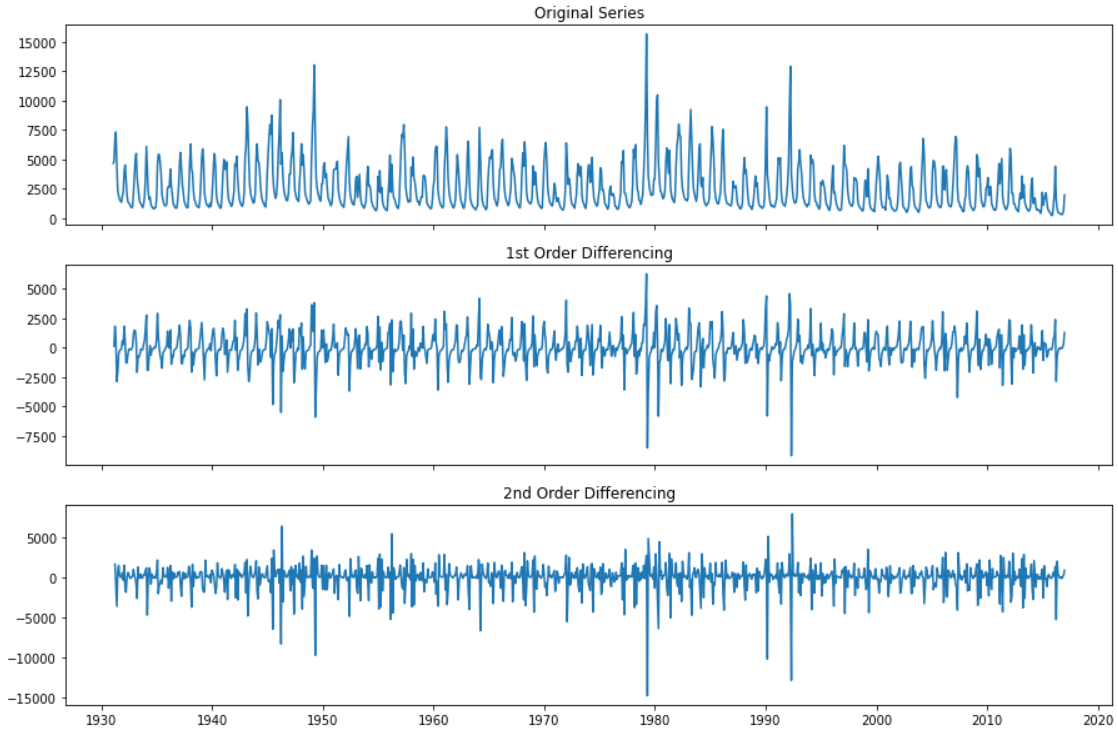
[ ]:
```
# d parameter
fig, (ax1, ax2, ax3) = plt.subplots(3)
ax1.plot(format.VALUE); ax1.set_title('Original Series')
ax1.axes.xaxis.set_visible(False)
# 1st Differencing
ax2.plot(format.VALUE.diff()); ax2.set_title('1st Order Differencing')
ax2.axes.xaxis.set_visible(False)
# 2nd Differencing
ax3.plot(format.VALUE.diff().diff()); ax3.set_title('2nd Order Differencing')
plt.show()
```

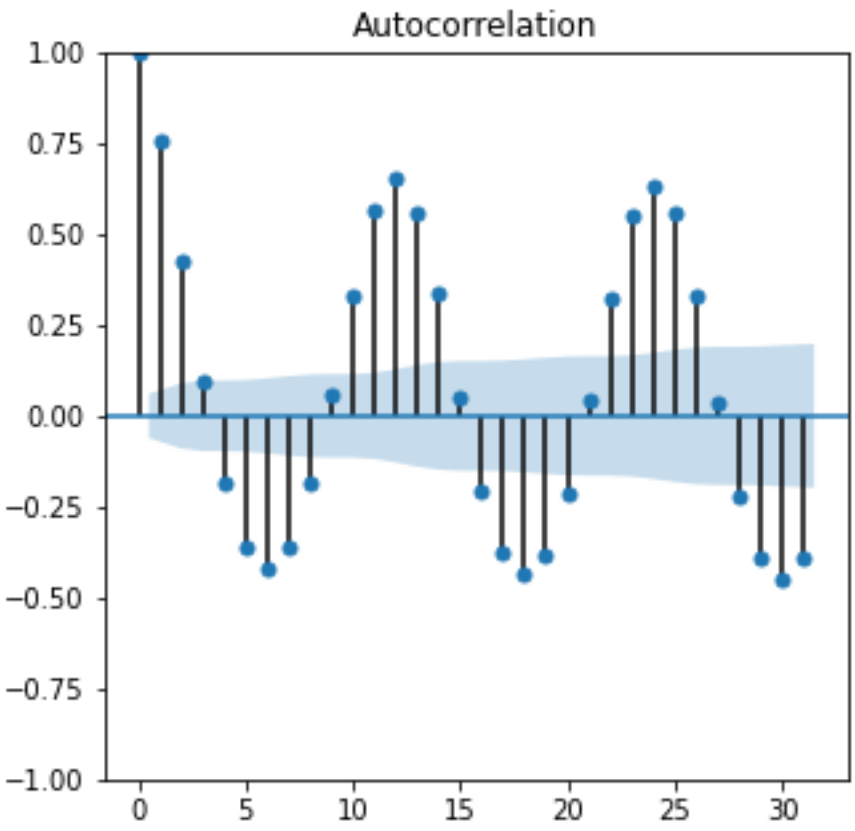


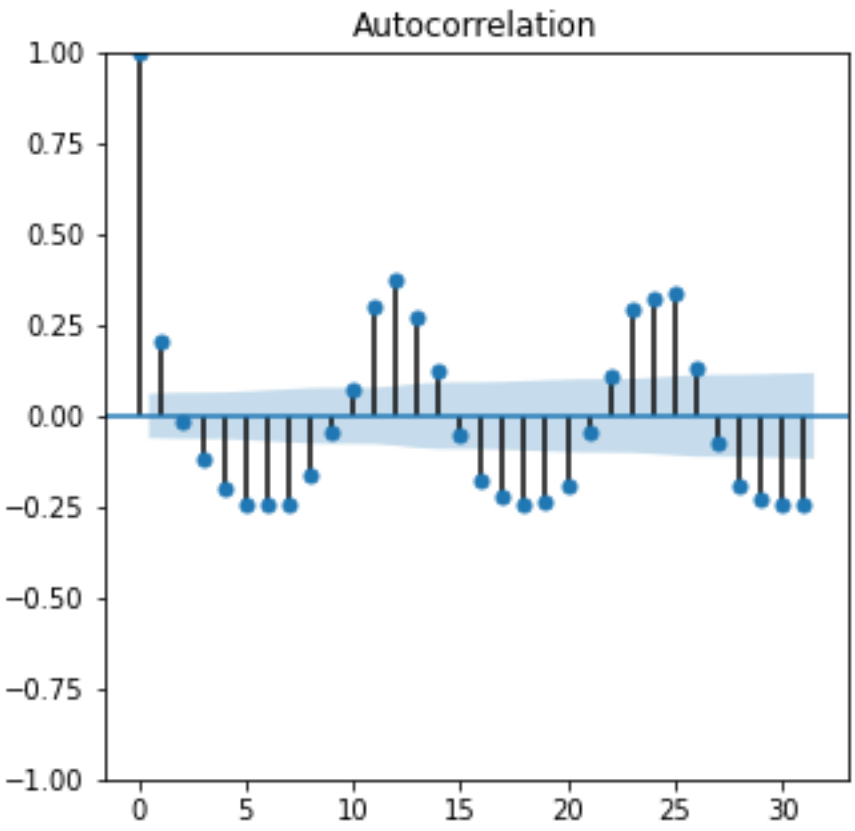
[ ]:
```
# p
plt.close()
# Original Series
fig, axes = plt.subplots(3, 1, sharex=True, figsize=(15,10))
axes[0].plot(format.VALUE)
axes[0].set_title('Original Series')
result = adfuller(format.VALUE.dropna())
print("p-value:",result[1])
plot_acf(format.VALUE)
# 1st Differencing
axes[1].plot(format.VALUE.diff())
axes[1].set_title('1st Order Differencing')
result = adfuller(format.VALUE.diff().dropna())
plot_acf(database.VALUE.diff().dropna())
print("p-value:",result[1])
# 2nd Differencing
axes[2].plot(format.VALUE.diff().diff())
axes[2].set_title('2nd Order Differencing')
plot_acf(database.VALUE.diff().diff().dropna())
result = adfuller(format.VALUE.diff().diff().dropna())
```

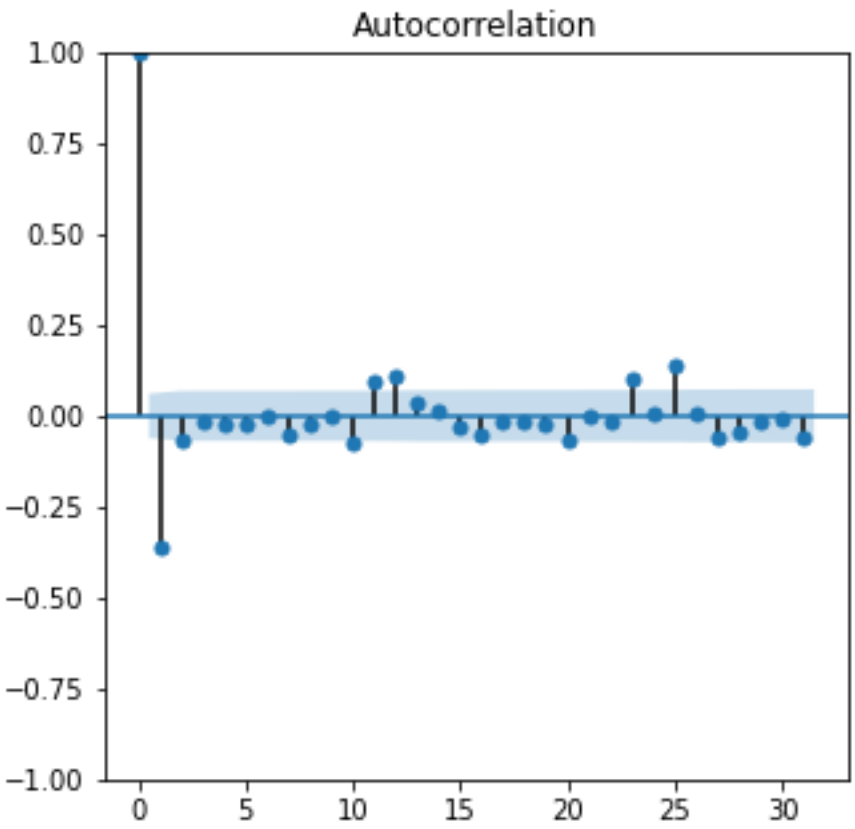
```
print("p-value:",result[1])
plt.show()
```

```
p-value: 0.02407952227509615
p-value: 1.4113310892830462e-23
p-value: 6.002347173917031e-23
```



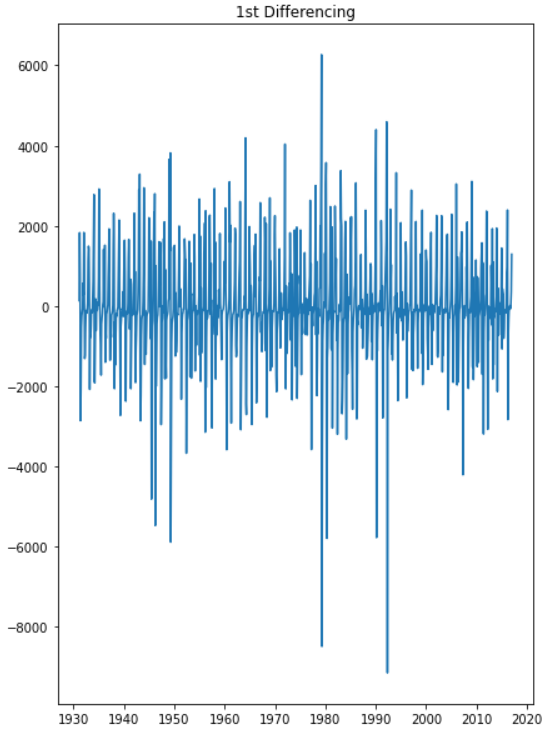

Autocorrelation
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30

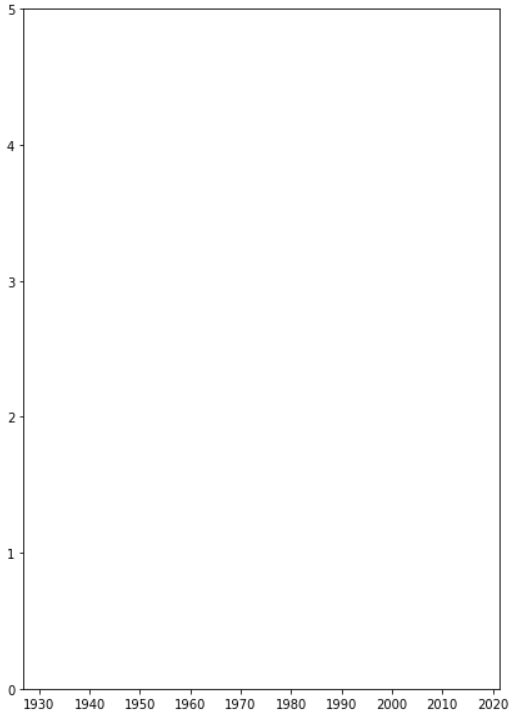

Autocorrelation
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30

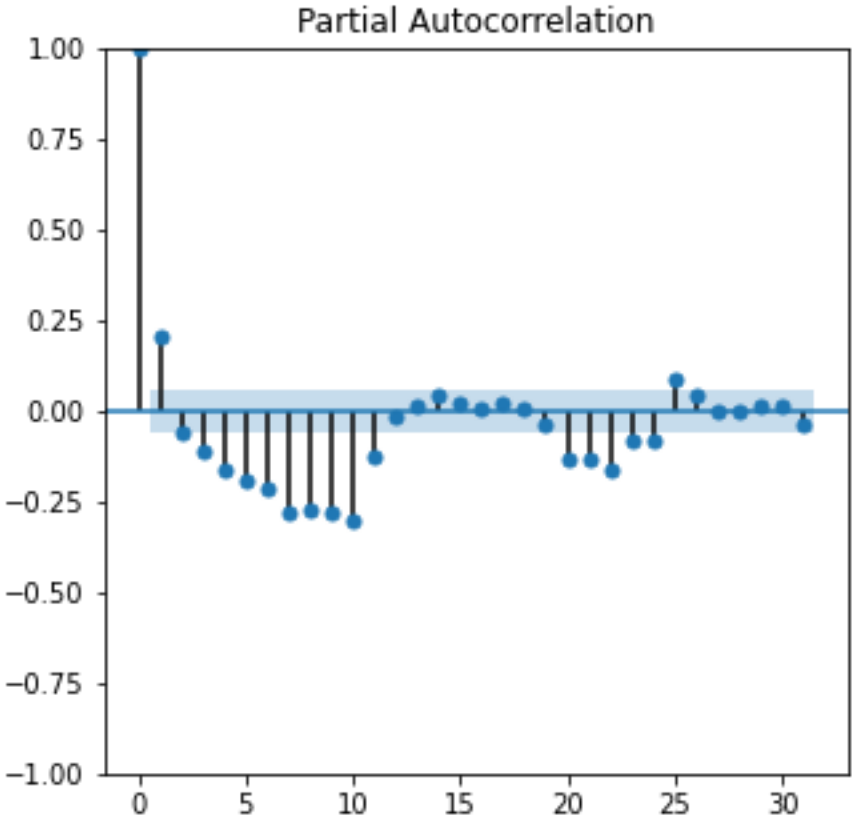


[ ]:
```
# d
fig, axes = plt.subplots(1,2, sharex=True, figsize=(15,10))
axes[0].plot(format.VALUE.diff())
axes[0].set_title('1st Differencing')
axes[1].set(ylim=(0,5))
plot_pacf(format.VALUE.diff().dropna())
plt.show()
```

```
/usr/local/lib/python3.7/dist-packages/statsmodels/graphics/tsaplots.py:353:
FutureWarning: The default method 'yw' can produce PACF values outside of the
[-1,1] interval. After 0.13, the default will change tounadjusted Yule-Walker
('ywm'). You can use this method now by setting method='ywm'.
  FutureWarning,
```

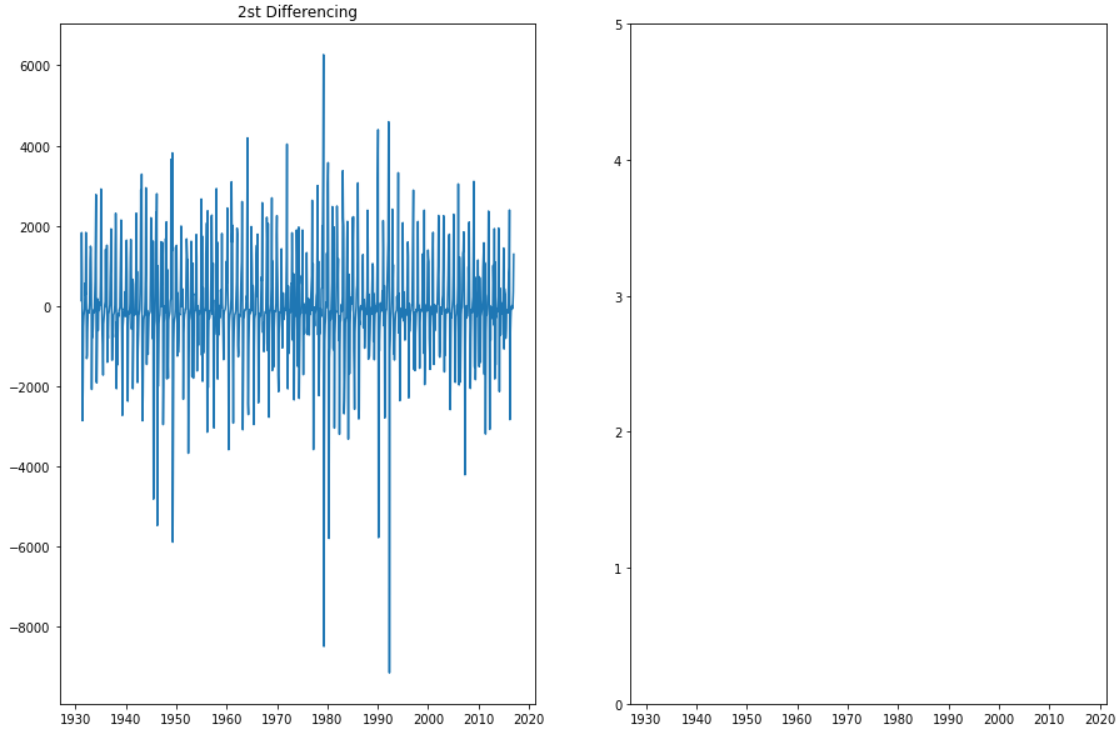

1st Differencing
6000
4000
2000
0
−2000
−4000
−6000
−8000
1930
1940
1950
1960
1970
1980
1990
2000
2010
2020

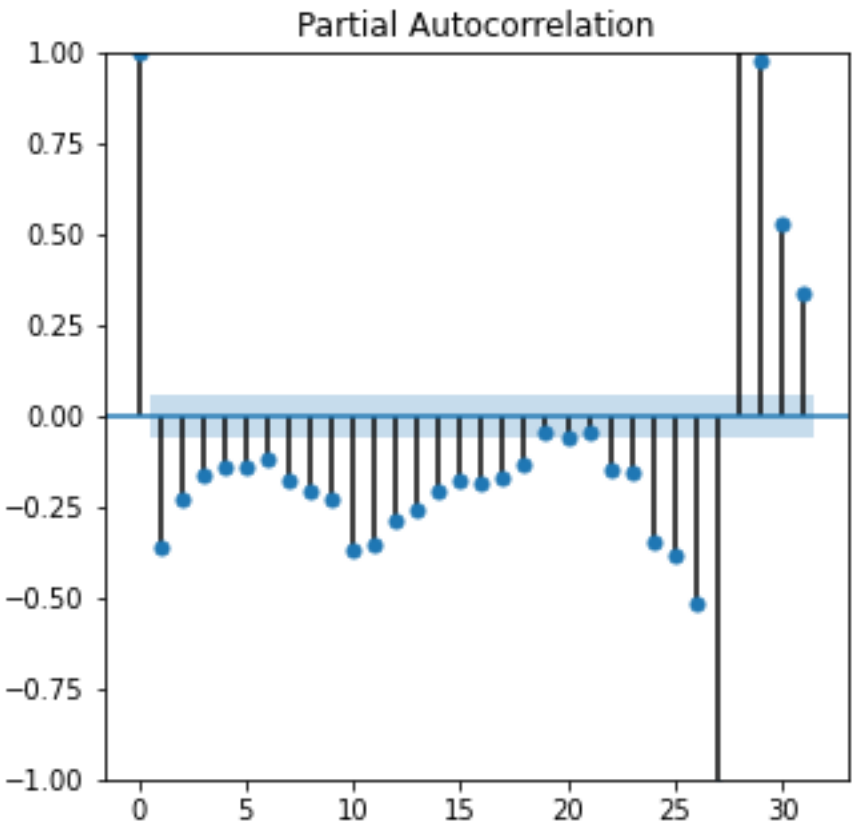

5
4
3
2
1
0
1930
1940
1950
1960
1970
1980
1990
2000
2010
2020


Partial Autocorrelation
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30

[ ]:
```
fig, axes = plt.subplots(1, 2, sharex=True, figsize=(15,10))
axes[0].plot(format.VALUE.diff())
axes[0].set_title('2st Differencing')
axes[1].set(ylim=(0,5))
plot_pacf(format.VALUE.diff().diff().dropna())
plt.show()
```

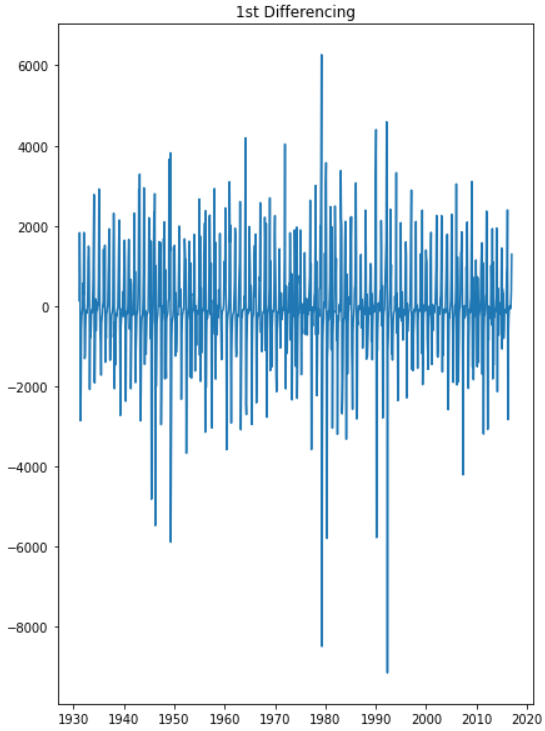
```
/usr/local/lib/python3.7/dist-packages/statsmodels/graphics/tsaplots.py:353:
FutureWarning: The default method 'yw' can produce PACF values outside of the
[-1,1] interval. After 0.13, the default will change tounadjusted Yule-Walker
('ywm'). You can use this method now by setting method='ywm'.
  FutureWarning,
```

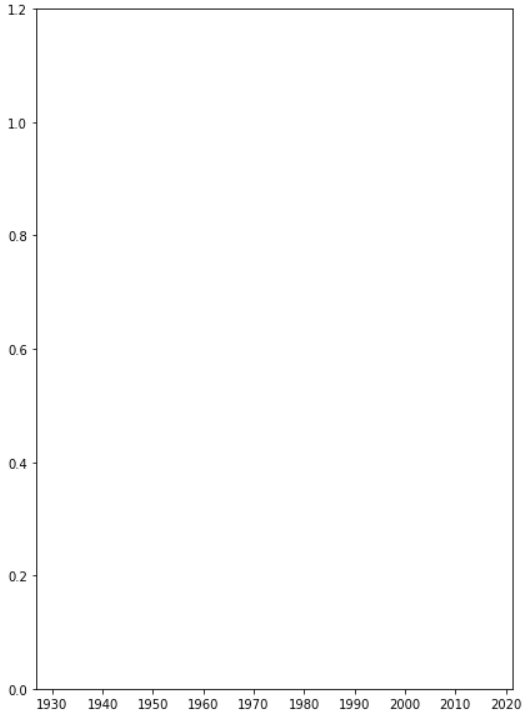


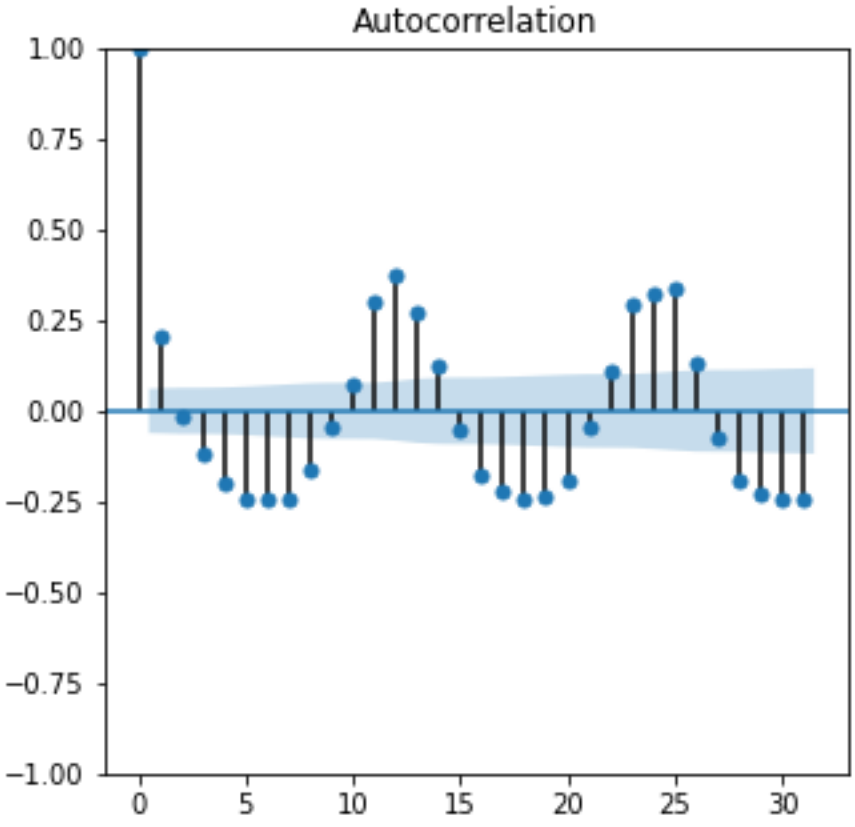


Como a segunda diferenciação do D criaram resultados fora do padrão esperado, usaremos somente a primeira diferenciação

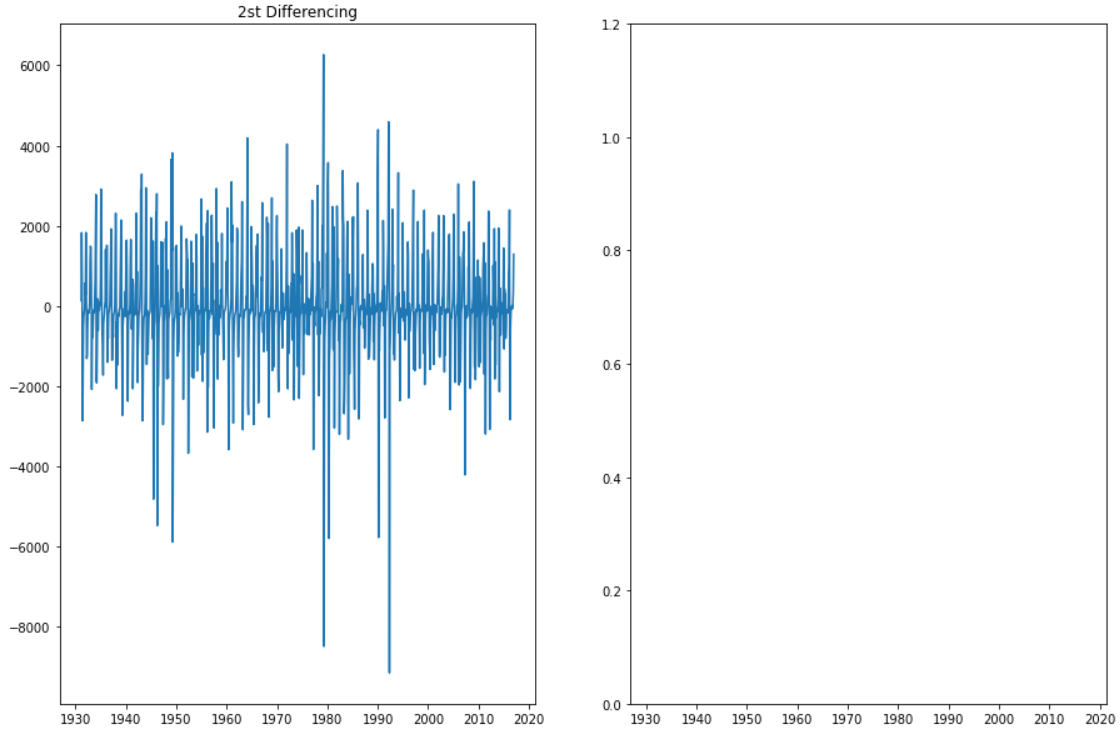
[ ]:
```
# q
fig, axes = plt.subplots(1, 2, sharex=True, figsize=(15,10))
axes[0].plot(format.VALUE.diff())
axes[0].set_title('1st Differencing')
axes[1].set(ylim=(0,1.2))
plot_acf(format.VALUE.diff().dropna())
plt.show()
```

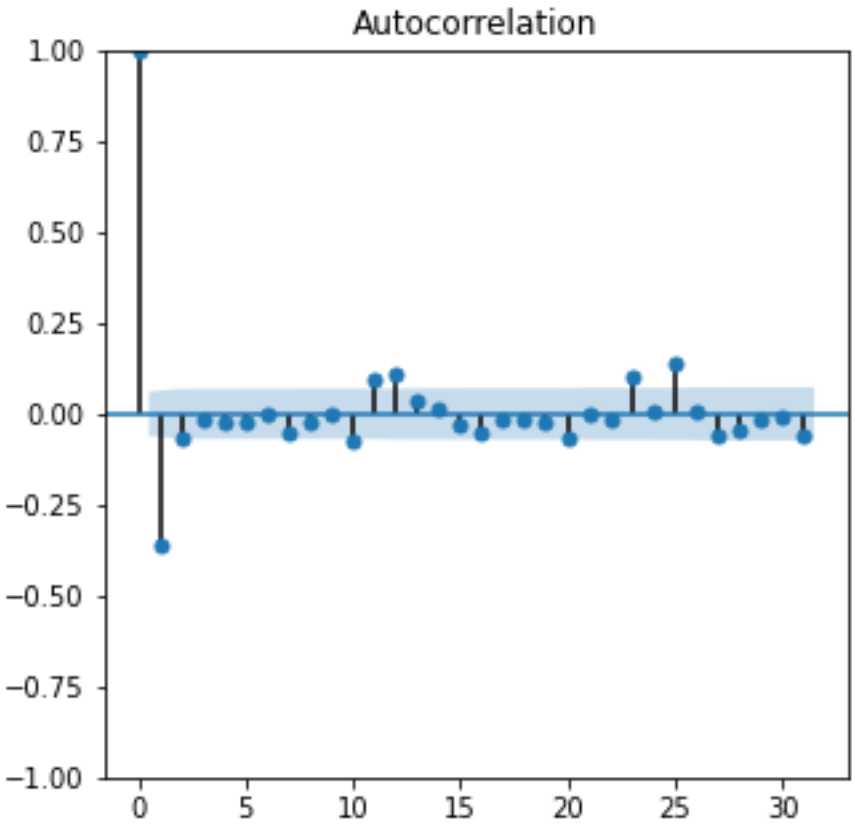

1st Differencing
6000
4000
2000
0
−2000
−4000
−6000
−8000
1930
1940
1950
1960
1970
1980
1990
2000
2010
2020


1.2
1.0
0.8
0.6
0.4
0.2
0.0
1930
1940
1950
1960
1970
1980
1990
2000
2010
2020


Autocorrelation
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30

[ ]:
```
# q
fig, axes = plt.subplots(1, 2, sharex=True, figsize=(15,10))
axes[0].plot(format.VALUE.diff())
axes[0].set_title('2st Differencing')
axes[1].set(ylim=(0,1.2))
plot_acf(format.VALUE.diff().diff().dropna())
plt.show()
```





Na segunda diferenciação do Q, os resultados obtidos foram desejaveis. Assim consideraremos ele como sendo de diferenciação 2

[ ]:
```
from statsmodels.tsa.arima.model import ARIMA
```

[ ]:
```
model = ARIMA(format.VALUE, order=(1,1,2))
model_fit = model.fit()
print(model_fit.summary())
```

```
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:471:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:471:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:471:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
```

```
/usr/local/lib/python3.7/dist-
packages/statsmodels/tsa/statespace/sarimax.py:966: UserWarning: Non-stationary
starting autoregressive parameters found. Using zeros as starting parameters.
  warn('Non-stationary starting autoregressive parameters'
/usr/local/lib/python3.7/dist-
packages/statsmodels/tsa/statespace/sarimax.py:978: UserWarning: Non-invertible
starting MA parameters found. Using zeros as starting parameters.
  warn('Non-invertible starting MA parameters found.'
```

SARIMAX Results

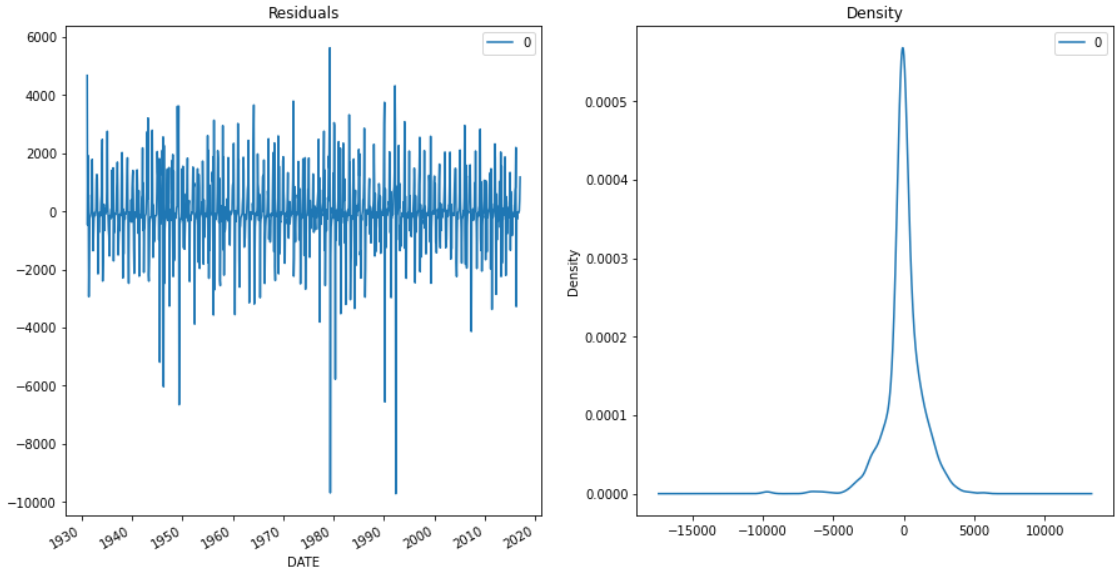
Dep. Variable:	VALUE	No. Observations:	1032
Model:	ARIMA(1, 1, 2)	Log Likelihood	-8864.366
Date:	Tue, 30 Aug 2022	AIC	17736.733
Time:	21:34:12	BIC	17756.486
Sample:	0	HQIC	17744.229
	- 1032		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	-0.9891	0.043	-22.769	0.000	-1.074	-0.904
ma.L1	1.1969	0.049	24.358	0.000	1.101	1.293
ma.L2	0.2078	0.023	9.163	0.000	0.163	0.252
sigma2	1.72e+06	3.92e+04	43.878	0.000	1.64e+06	1.8e+06

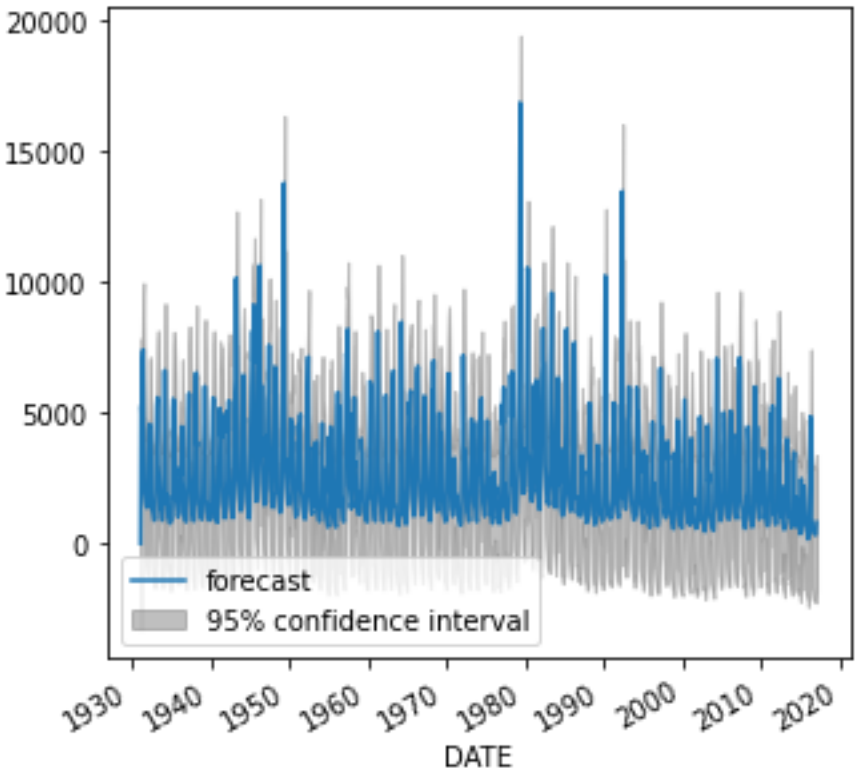
Ljung-Box (L1) (Q):	0.02	Jarque-Bera (JB):	3083.00
Prob(Q):	0.88	Prob(JB):	0.00
Heteroskedasticity (H):	0.92	Skew:	-1.13
Prob(H) (two-sided):	0.46	Kurtosis:	11.17

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

[ ]:
```
residuals = pd.DataFrame(model_fit.resid)
fig, ax = plt.subplots(1,2, figsize=(15,8))
residuals.plot(title="Residuals", ax=ax[0])
residuals.plot(kind='kde', title='Density', ax=ax[1])
plt.show()
```



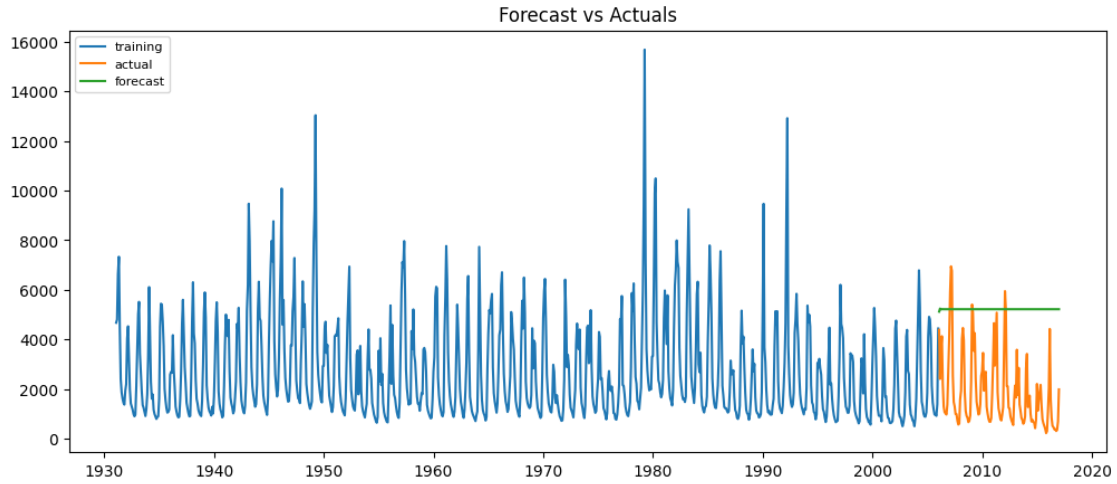
[ ]:
```
from statsmodels.graphics.tsaplots import plot_predict
plot_predict(model_fit,dynamic=False)
plt.show()
```



[ ]:
```
# Create Training and Test
train = database.VALUE[:900]
test = database.VALUE[900:]
```

[ ]:
```
model = ARIMA(train, order=(1, 1, 2))
fitted = model.fit()
# Forecast
fc = fitted.forecast(132, alpha=0.05,h=30) # 95% conf
conf = fitted.conf_int()
se = fitted.bse
# fc, se, conf = fitted.forecast(24, alpha=0.05) # 95% conf
```

[ ]:
```
# Make as pandas series
fc_series = pd.Series(fc.values, index=test.index)
lower_series = pd.Series(conf[0], index=test.index)
upper_series = pd.Series(conf[1], index=test.index)
# Plot
plt.figure(figsize=(12,5), dpi=100)
plt.plot(train, label='training')
plt.plot(test, label='actual')
plt.plot(fc_series, label='forecast')
plt.fill_between(lower_series.index, lower_series, upper_series, color='k',␣
   alpha=.15)
plt.title('Forecast vs Actuals')
plt.legend(loc='upper left', fontsize=8)
plt.show()
```



[ ]:
```
model1 = ARIMA(train, order=(1, 1, 2))
fitted = model1.fit()
print(fitted.summary())
# Forecast
fc = fitted.forecast(132, alpha=0.05) # 95% conf
conf = fitted.conf_int()
se = fitted.bse
# fc, se, conf = fitted.forecast(24, alpha=0.05) # 95% conf
```

```
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:471:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:471:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:471:
ValueWarning: A date index has been provided, but it has no associated frequency
information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.7/dist-
packages/statsmodels/tsa/statespace/sarimax.py:966: UserWarning: Non-stationary
starting autoregressive parameters found. Using zeros as starting parameters.
  warn('Non-stationary starting autoregressive parameters'
/usr/local/lib/python3.7/dist-
packages/statsmodels/tsa/statespace/sarimax.py:978: UserWarning: Non-invertible
starting MA parameters found. Using zeros as starting parameters.
  warn('Non-invertible starting MA parameters found.'
```

SARIMAX Results

Dep. Variable:	VALUE	No. Observations:	900
Model:	ARIMA(1, 1, 2)	Log Likelihood	-7747.500
Date:	Tue, 30 Aug 2022	AIC	15503.000
Time:	21:34:14	BIC	15522.205
Sample:	0	HQIC	15510.337
	- 900		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	-0.2499	0.848	-0.295	0.768	-1.912	1.412
ma.L1	0.4744	0.849	0.559	0.576	-1.189	2.138
ma.L2	0.0995	0.189	0.527	0.598	-0.271	0.470
sigma2	1.79e+06	5.1e+04	35.128	0.000	1.69e+06	1.89e+06

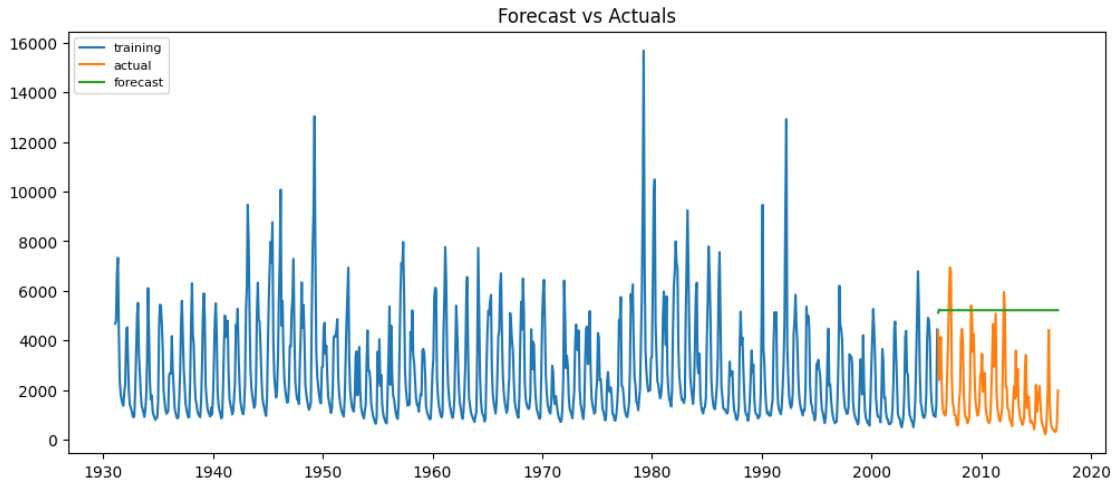
```
===
```

```
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):
3313.84
Prob(Q):                              0.98   Prob(JB):
0.00
Heteroskedasticity (H):               1.06   Skew:
-1.27
Prob(H) (two-sided):                  0.60   Kurtosis:
12.06
================================================================================
===

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).

/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:843:
ValueWarning: No supported index is available. Prediction results will be given
with an integer index beginning at `start`.
  data=self.data,
```

[ ]:
```
# Make as pandas series
fc_series = pd.Series(fc.values, index=test.index)
lower_series = pd.Series(conf[0], index=test.index)
upper_series = pd.Series(conf[1], index=test.index)
# Plot
plt.figure(figsize=(12,5), dpi=100)
plt.plot(train, label='training')
plt.plot(test, label='actual')
plt.plot(fc_series, label='forecast')
plt.fill_between(lower_series.index, lower_series, upper_series, color='k',␣
  alpha=.15)
plt.title('Forecast vs Actuals')
plt.legend(loc='upper left', fontsize=8)
plt.show()
```



[ ]:
```
def forecast_accuracy(fcast, atual):
  mape = np.mean(np.abs(fcast - atual)/np.abs(atual)) # MAPE
  me = np.mean(fcast - atual) # ME
  mae = np.mean(np.abs(fcast - atual)) # MAE
  mpe = np.mean((fcast - atual)/atual) # MPE
  rmse = np.mean((fcast - atual)**2)**.5 # RMSE
  corr = np.corrcoef(fcast, atual)[0,1] # corr
  mins = np.amin(np.hstack([fcast[:,None],
  atual[:,None]]), axis=1)
  maxs = np.amax(np.hstack([fcast[:,None],
  atual[:,None]]), axis=1)
  minmax = 1 - np.mean(mins/maxs) # minmax
  acf1 = acf(fc-test)[1] # ACF1
  return({'mape':mape, 'me':me, 'mae': mae,
        'mpe': mpe, 'rmse':rmse, 'acf1':acf1,
        'corr':corr, 'minmax':minmax})

forecast_accuracy(fc, test.values)
```

```
/usr/local/lib/python3.7/dist-packages/ipykernel_launcher.py:8: FutureWarning:
Support for multi-dimensional indexing (e.g. `obj[:, None]`) is deprecated and
will be removed in a future version.  Convert to a numpy array before indexing
instead.

/usr/local/lib/python3.7/dist-packages/ipykernel_launcher.py:10: FutureWarning:
Support for multi-dimensional indexing (e.g. `obj[:, None]`) is deprecated and
will be removed in a future version.  Convert to a numpy array before indexing
instead.
  # Remove the CWD from sys.path while we load stuff.
```

[ ]:
```
{'mape': 3.8316720870337098,
 'me': 3331.2272309945547,
 'mae': 3405.057975670473,
 'mpe': 3.820306534270911,
 'rmse': 3647.358758901422,
 'acf1': nan,
 'corr': -0.13912266153610595,
 'minmax': 0.6508955296530141}
```

[ ]:
```
model = pm.auto_arima(database.VALUE,start_p=1,start_q=1,
test='adf',
max_p=3, max_q=3, m=12,
start_P=0, seasonal=False,
d=None, D=0, trace=True,
error_action='ignore',
suppress_warnings=True,
stepwise=True)
model.summary()
```

```
/usr/local/lib/python3.7/dist-packages/pmdarima/arima/_validation.py:62:
UserWarning: m (12) set for non-seasonal fit. Setting to 0
  warnings.warn("m (%i) set for non-seasonal fit. Setting to 0" % m)

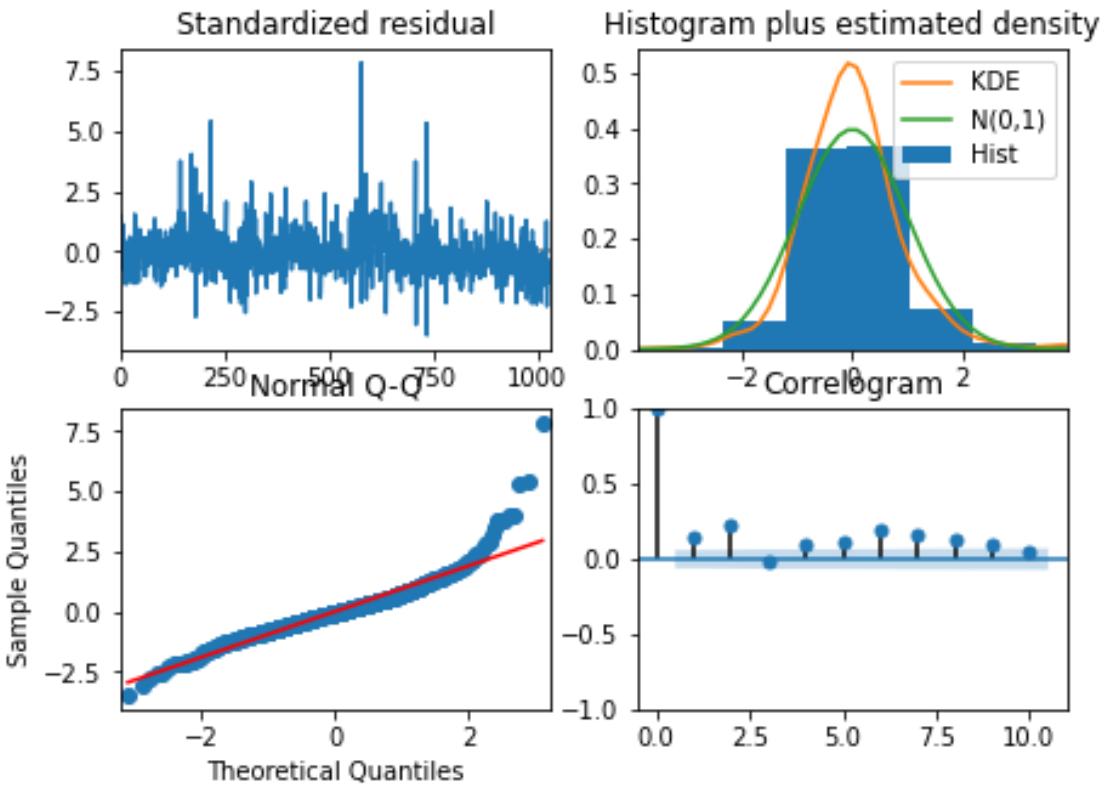
Performing stepwise search to minimize aic
 ARIMA(1,0,1)(0,0,0)[0]             : AIC=17690.293, Time=0.12 sec
 ARIMA(0,0,0)(0,0,0)[0]             : AIC=19611.515, Time=0.03 sec
 ARIMA(1,0,0)(0,0,0)[0]             : AIC=17752.248, Time=0.05 sec
 ARIMA(0,0,1)(0,0,0)[0]             : AIC=18646.954, Time=0.22 sec
 ARIMA(2,0,1)(0,0,0)[0]             : AIC=17684.901, Time=0.15 sec
 ARIMA(2,0,0)(0,0,0)[0]             : AIC=17683.002, Time=0.09 sec
 ARIMA(3,0,0)(0,0,0)[0]             : AIC=17684.859, Time=0.11 sec
 ARIMA(3,0,1)(0,0,0)[0]             : AIC=17686.634, Time=0.17 sec
 ARIMA(2,0,0)(0,0,0)[0] intercept   : AIC=17518.526, Time=0.16 sec
 ARIMA(1,0,0)(0,0,0)[0] intercept   : AIC=17666.698, Time=0.10 sec
 ARIMA(3,0,0)(0,0,0)[0] intercept   : AIC=17483.248, Time=0.24 sec
 ARIMA(3,0,1)(0,0,0)[0] intercept   : AIC=17478.187, Time=0.61 sec
 ARIMA(2,0,1)(0,0,0)[0] intercept   : AIC=17515.551, Time=0.57 sec
 ARIMA(3,0,2)(0,0,0)[0] intercept   : AIC=17461.027, Time=2.14 sec
 ARIMA(2,0,2)(0,0,0)[0] intercept   : AIC=17498.776, Time=0.77 sec
 ARIMA(3,0,3)(0,0,0)[0] intercept   : AIC=17457.624, Time=2.10 sec
 ARIMA(2,0,3)(0,0,0)[0] intercept   : AIC=17344.795, Time=2.27 sec
 ARIMA(1,0,3)(0,0,0)[0] intercept   : AIC=17539.531, Time=0.36 sec
 ARIMA(1,0,2)(0,0,0)[0] intercept   : AIC=17552.108, Time=0.25 sec
 ARIMA(2,0,3)(0,0,0)[0]             : AIC=17693.099, Time=0.56 sec

Best model:  ARIMA(2,0,3)(0,0,0)[0] intercept
Total fit time: 11.098 seconds
```

[ ]:
```
<class 'statsmodels.iolib.summary.Summary'>
"""
                               SARIMAX Results
==========================================================================
Dep. Variable:                      y   No. Observations:                 1032
Model:               SARIMAX(2, 0, 3)   Log Likelihood               -8665.397
Date:                Tue, 30 Aug 2022   AIC                          17344.795
Time:                        21:34:26   BIC                          17379.370
Sample:                             0   HQIC                         17357.916
                                 - 1032
Covariance Type:                  opg
==========================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
--------------------------------------------------------------------------
intercept    692.4821     23.229     29.811      0.000     646.953     738.011
ar.L1          1.7312      0.002    854.087      0.000       1.727       1.735
ar.L2         -0.9994      0.001   -885.196      0.000      -1.002      -0.997
ma.L1         -1.1041      0.022    -50.349      0.000      -1.147      -1.061
ma.L2          0.0242      0.040      0.608      0.543      -0.054       0.102
ma.L3          0.5216      0.028     18.757      0.000       0.467       0.576
sigma2      1.255e+06      0.002   6.62e+08      0.000    1.25e+06    1.25e+06
==========================================================================
===
Ljung-Box (L1) (Q):                  22.25   Jarque-Bera (JB):
2700.36
Prob(Q):                              0.00   Prob(JB):
0.00
Heteroskedasticity (H):               0.92   Skew:
1.28
Prob(H) (two-sided):                  0.43   Kurtosis:
10.50
==========================================================================
===

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
[2] Covariance matrix is singular or near-singular, with condition number
7.71e+23. Standard errors may be unstable.
"""
```

[ ]:
```
model.plot_diagnostics(figsize=(7,5))
plt.show()
```

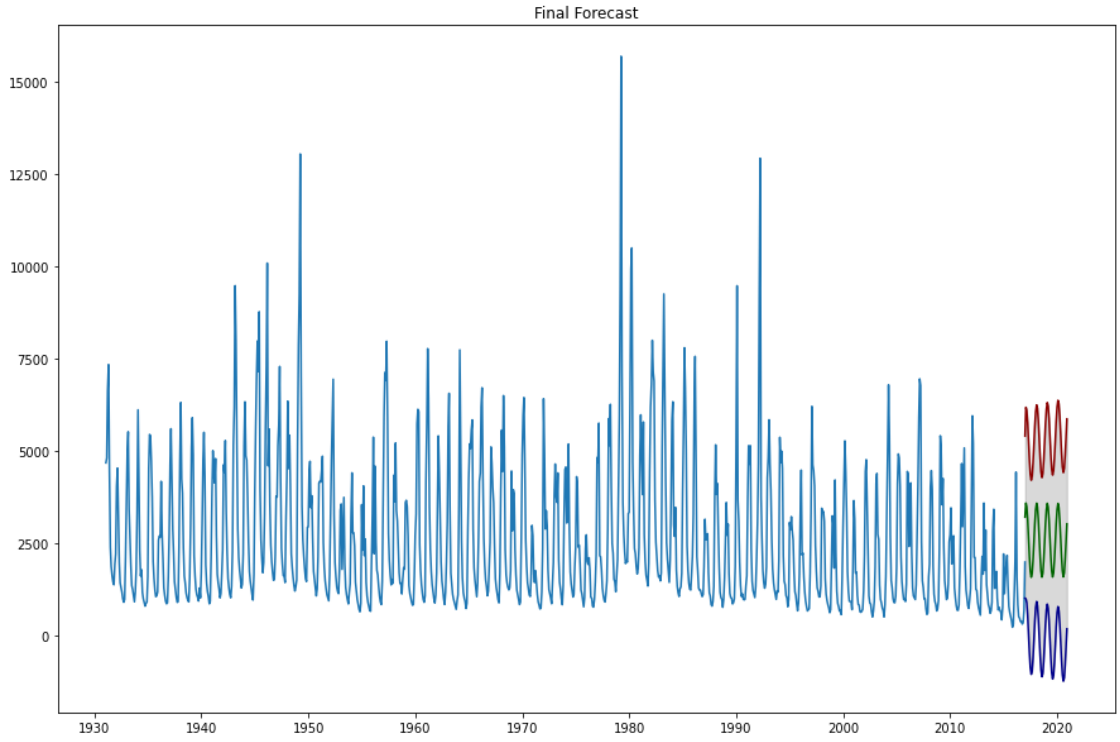


[ ]:
```
from matplotlib.pyplot import figure
from datetime import timedelta
```

[ ]:
```
figure(figsize=(15,10))
# Forecast
n_periods = 48
fc, confint = model.predict(n_periods=n_periods, return_conf_int=True)
index_of_fc = np.arange(len(database.VALUE), len(database.VALUE)+n_periods)
index_of_fc = pd.date_range(database.index[-1],freq='MS',periods=48)
# make series for plotting purpose
fc_series = pd.Series(fc.values, index=index_of_fc)
lower_series = pd.Series(confint[:, 0], index=index_of_fc)
upper_series = pd.Series(confint[:, 1], index=index_of_fc)
# Plot
plt.plot(database.VALUE)
plt.plot(fc_series, color='darkgreen')
plt.plot(upper_series, color='darkred')
plt.plot(lower_series, color='darkblue')
plt.fill_between(lower_series.index, lower_series, upper_series, color='k',␣
   alpha=.15)
plt.title("Final Forecast")
```

```
plt.show()
```

```
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:843:
ValueWarning: No supported index is available. Prediction results will be given
with an integer index beginning at `start`.
  data=self.data,
```



[ ]:
```
from dateutil.parser import parse
```

[ ]:
```
database.drop('year', axis=1, inplace=True)
database.drop('month',axis=1,inplace=True)
database
```

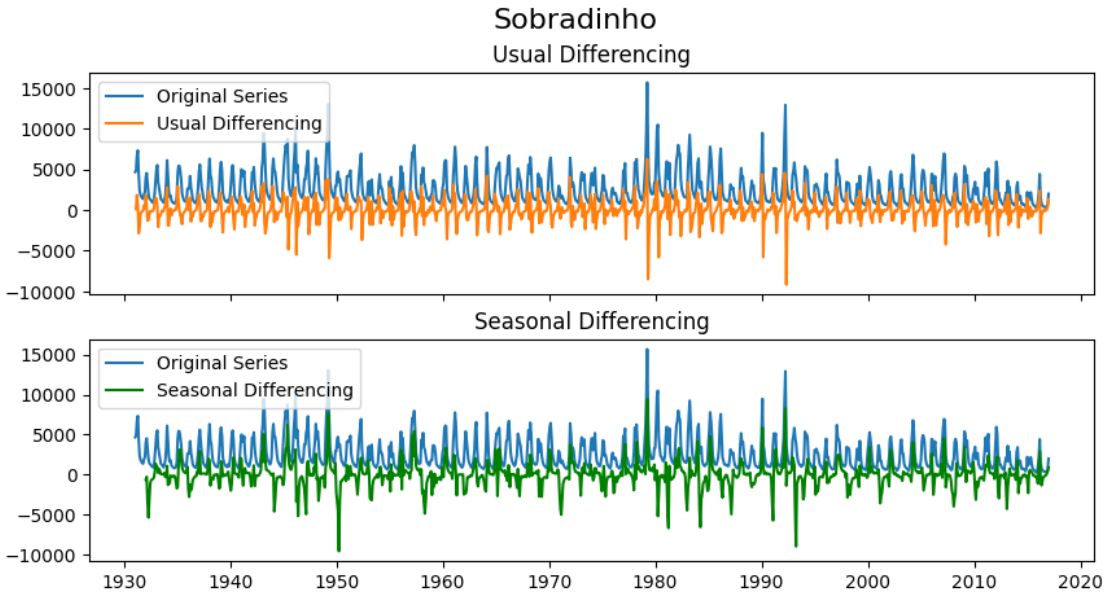
[ ]:
```
             VALUE
DATE
1931-01-30  4676.0
1931-02-28  4805.0
1931-03-30  6630.0
1931-04-30  7333.0
1931-05-30  4467.0
…            …
2016-08-30   386.0
2016-09-30   313.0
```

```
2016-10-30   354.0
2016-11-30   704.0
2016-12-30  1989.0

[1032 rows x 1 columns]
```

[ ]:
```
# Plot
fig, axes = plt.subplots(2, 1, figsize=(10,5), dpi=100, sharex=True)
# Usual Differencing
axes[0].plot(format[:], label='Original Series')
axes[0].plot(format[:].diff(1), label='Usual Differencing')
axes[0].set_title('Usual Differencing')
axes[0].legend(loc='upper left', fontsize=10)

# Seasinal Dei
axes[1].plot(format[:], label='Original Series')
axes[1].plot(format[:].diff(12), label='Seasonal Differencing', color='green')
axes[1].set_title('Seasonal Differencing')
plt.legend(loc='upper left', fontsize=10)
plt.suptitle('Sobradinho', fontsize=16)
plt.show()
```



[ ]:
```
smodel = pm.auto_arima(format.VALUE, start_p=1, start_q=1,
test='adf', # use adftest to find optimal 'd'
max_p=3, max_q=3, # maximum p and q
m=12, # frequency of series
d=1,
```

```
seasonal=True,
start_P=0,
D=1,
trace=True,
error_action='ignore',
suppress_warnings=True,
stepwise=True)
print(smodel.summary())
```

```
Performing stepwise search to minimize aic
 ARIMA(1,1,1)(0,1,1)[12]             : AIC=inf, Time=9.55 sec
 ARIMA(0,1,0)(0,1,0)[12]             : AIC=17788.557, Time=0.08 sec
 ARIMA(1,1,0)(1,1,0)[12]             : AIC=17532.345, Time=2.55 sec
 ARIMA(0,1,1)(0,1,1)[12]             : AIC=inf, Time=2.83 sec
 ARIMA(1,1,0)(0,1,0)[12]             : AIC=17769.650, Time=0.14 sec
 ARIMA(1,1,0)(2,1,0)[12]             : AIC=17351.646, Time=7.17 sec
 ARIMA(1,1,0)(2,1,1)[12]             : AIC=inf, Time=13.40 sec
 ARIMA(1,1,0)(1,1,1)[12]             : AIC=inf, Time=4.23 sec
 ARIMA(0,1,0)(2,1,0)[12]             : AIC=17368.561, Time=1.80 sec
 ARIMA(2,1,0)(2,1,0)[12]             : AIC=17306.745, Time=14.32 sec
 ARIMA(2,1,0)(1,1,0)[12]             : AIC=17482.343, Time=3.92 sec
 ARIMA(2,1,0)(2,1,1)[12]             : AIC=inf, Time=13.99 sec
 ARIMA(2,1,0)(1,1,1)[12]             : AIC=inf, Time=5.51 sec
 ARIMA(3,1,0)(2,1,0)[12]             : AIC=17279.109, Time=10.29 sec
 ARIMA(3,1,0)(1,1,0)[12]             : AIC=17448.391, Time=4.45 sec
 ARIMA(3,1,0)(2,1,1)[12]             : AIC=inf, Time=16.37 sec
 ARIMA(3,1,0)(1,1,1)[12]             : AIC=inf, Time=6.64 sec
 ARIMA(3,1,1)(2,1,0)[12]             : AIC=inf, Time=18.88 sec
 ARIMA(2,1,1)(2,1,0)[12]             : AIC=inf, Time=21.97 sec
 ARIMA(3,1,0)(2,1,0)[12] intercept   : AIC=17281.117, Time=11.60 sec

Best model:  ARIMA(3,1,0)(2,1,0)[12]
Total fit time: 169.740 seconds
                                      SARIMAX Results
==========================================================================
==========
Dep. Variable:                                     y   No. Observations:
1032
Model:             SARIMAX(3, 1, 0)x(2, 1, 0, 12)   Log Likelihood
-8633.554
Date:                            Tue, 30 Aug 2022   AIC
17279.109
Time:                                    21:37:17   BIC
17308.668
Sample:                                          0   HQIC
17290.333
                                             - 1032
```

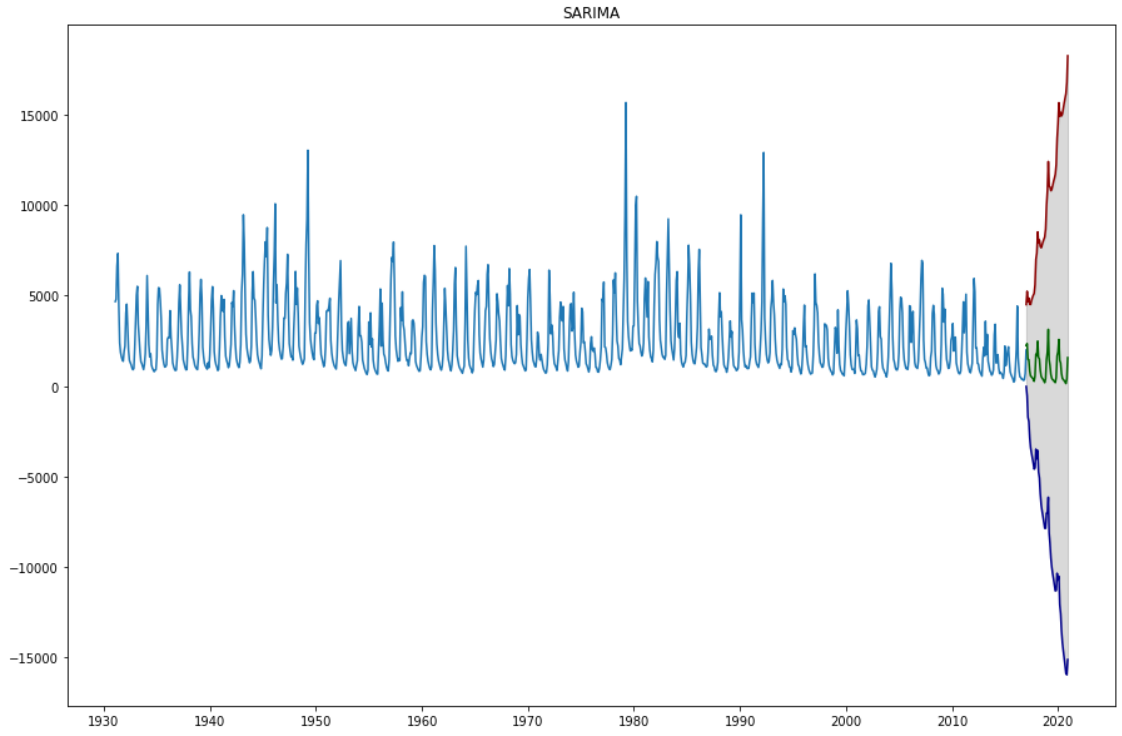
```
Covariance Type:                                 opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.2027      0.017    -11.940      0.000      -0.236      -0.169
ar.L2         -0.2418      0.021    -11.564      0.000      -0.283      -0.201
ar.L3         -0.1698      0.029     -5.926      0.000      -0.226      -0.114
ar.S.L12      -0.6517      0.018    -36.461      0.000      -0.687      -0.617
ar.S.L24      -0.3959      0.018    -21.919      0.000      -0.431      -0.360
sigma2      1.336e+06   3.09e+04     43.264      0.000    1.28e+06     1.4e+06
==============================================================================
===
Ljung-Box (L1) (Q):                   0.83   Jarque-Bera (JB):
1736.02
Prob(Q):                              0.36   Prob(JB):
0.00
Heteroskedasticity (H):               0.90   Skew:
0.26
Prob(H) (two-sided):                  0.36   Kurtosis:
9.37
==============================================================================
===

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
```

[ ]:
```
figure(figsize=(15,10))
n_periods = 48
fitted, confint = smodel.predict(n_periods=n_periods, return_conf_int=True)
index_of_fc = pd.date_range(format.index[-1], periods = n_periods, freq='MS')
# make series for plotting purpose
fitted_series = pd.Series(fitted.values, index=index_of_fc)
lower_series = pd.Series(confint[:, 0], index=index_of_fc)
upper_series = pd.Series(confint[:, 1], index=index_of_fc)
# Plot
plt.plot(format)
plt.plot(fitted_series, color='darkgreen')
plt.plot(upper_series, color='darkred')
plt.plot(lower_series, color='darkblue')
plt.fill_between(lower_series.index, lower_series, upper_series, color='k',␣
  alpha=.15)
plt.title("SARIMA")
plt.show()
```

```
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:843:
ValueWarning: No supported index is available. Prediction results will be given
with an integer index beginning at `start`.
```

```
    data=self.data,
```



[ ]:
```
result_mul = seasonal_decompose(format['VALUE'], model='multiplicative',␣
   extrapolate_trend='freq',period = int(len(format)/2))
# period = int(len(format)/2)
seasonal_index = result_mul.seasonal[-12:].to_frame()
seasonal_index['month'] = pd.to_datetime(seasonal_index.index).month
# merge with the base data
format['month'] = format.index.month
data = pd.merge(format, seasonal_index, how='left', on='month')
data.columns = ['VALUE', 'month', 'seasonal_index']
data.index = data.index # reassign the index.
data
```

[ ]:

	VALUE	month	seasonal_index
0	4676.0	1	1.343092
1	4805.0	2	1.709146
2	6630.0	3	1.044065
3	7333.0	4	1.030525
4	4467.0	5	0.509010
…	…	…	…
1027	386.0	8	0.278022
1028	313.0	9	0.230382
1029	354.0	10	0.337273

```
1030   704.0     11        0.781195
1031  1989.0     12        1.311463

[1032 rows x 3 columns]
```

[ ]:
```
sxmodel = pm.auto_arima(format[['VALUE']],
start_p=1, start_q=1,
test='adf',
max_p=3, max_q=3, m=12,
start_P=0, seasonal=True,
d=None, D=1, trace=True,
error_action='ignore',
suppress_warnings=True,
stepwise=True)
sxmodel.summary()
```

```
Performing stepwise search to minimize aic
 ARIMA(1,0,1)(0,1,1)[12] intercept   : AIC=inf, Time=8.51 sec
 ARIMA(0,0,0)(0,1,0)[12] intercept   : AIC=17942.097, Time=0.09 sec
 ARIMA(1,0,0)(1,1,0)[12] intercept   : AIC=17314.989, Time=3.08 sec
 ARIMA(0,0,1)(0,1,1)[12] intercept   : AIC=inf, Time=3.37 sec
 ARIMA(0,0,0)(0,1,0)[12]             : AIC=17940.443, Time=0.07 sec
 ARIMA(1,0,0)(0,1,0)[12] intercept   : AIC=17558.308, Time=0.16 sec
 ARIMA(1,0,0)(2,1,0)[12] intercept   : AIC=17155.611, Time=8.33 sec
 ARIMA(1,0,0)(2,1,1)[12] intercept   : AIC=inf, Time=21.35 sec
 ARIMA(1,0,0)(1,1,1)[12] intercept   : AIC=inf, Time=8.74 sec
 ARIMA(0,0,0)(2,1,0)[12] intercept   : AIC=17574.534, Time=11.99 sec
 ARIMA(2,0,0)(2,1,0)[12] intercept   : AIC=17150.127, Time=9.46 sec
 ARIMA(2,0,0)(1,1,0)[12] intercept   : AIC=17310.039, Time=4.03 sec
 ARIMA(2,0,0)(2,1,1)[12] intercept   : AIC=17088.783, Time=28.18 sec
 ARIMA(2,0,0)(1,1,1)[12] intercept   : AIC=inf, Time=10.81 sec
 ARIMA(2,0,0)(2,1,2)[12] intercept   : AIC=inf, Time=26.76 sec
 ARIMA(2,0,0)(1,1,2)[12] intercept   : AIC=inf, Time=24.65 sec
 ARIMA(3,0,0)(2,1,1)[12] intercept   : AIC=inf, Time=28.88 sec
 ARIMA(2,0,1)(2,1,1)[12] intercept   : AIC=17083.339, Time=54.47 sec
 ARIMA(2,0,1)(1,1,1)[12] intercept   : AIC=17100.260, Time=11.41 sec
 ARIMA(2,0,1)(2,1,0)[12] intercept   : AIC=17151.797, Time=26.85 sec
 ARIMA(2,0,1)(2,1,2)[12] intercept   : AIC=inf, Time=30.17 sec
 ARIMA(2,0,1)(1,1,0)[12] intercept   : AIC=17311.868, Time=10.00 sec
 ARIMA(2,0,1)(1,1,2)[12] intercept   : AIC=inf, Time=34.48 sec
 ARIMA(1,0,1)(2,1,1)[12] intercept   : AIC=inf, Time=27.68 sec
 ARIMA(3,0,1)(2,1,1)[12] intercept   : AIC=inf, Time=36.73 sec
 ARIMA(2,0,2)(2,1,1)[12] intercept   : AIC=inf, Time=41.42 sec
 ARIMA(1,0,2)(2,1,1)[12] intercept   : AIC=inf, Time=27.46 sec
 ARIMA(3,0,2)(2,1,1)[12] intercept   : AIC=inf, Time=37.05 sec
 ARIMA(2,0,1)(2,1,1)[12]             : AIC=inf, Time=20.18 sec
```

```
Best model:  ARIMA(2,0,1)(2,1,1)[12] intercept
Total fit time: 556.407 seconds
```

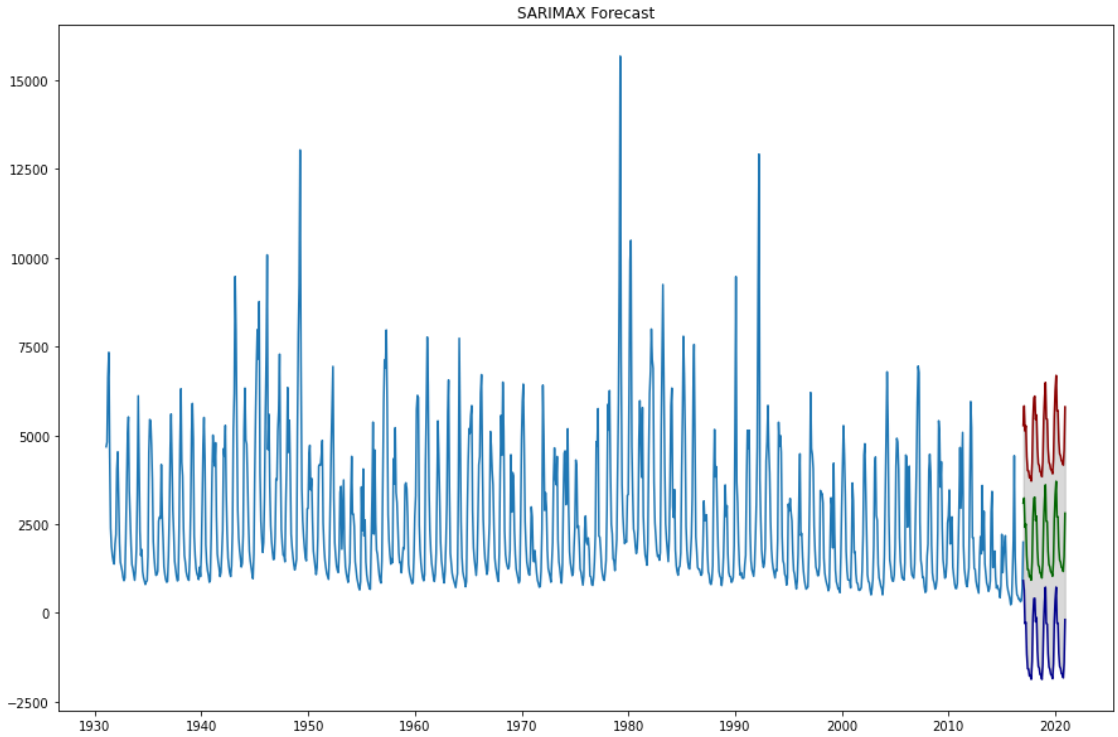
[ ]:
```
<class 'statsmodels.iolib.summary.Summary'>
"""
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                                  y   No. Observations:
1032
Model:             SARIMAX(2, 0, 1)x(2, 1, 1, 12)   Log Likelihood
-8533.669
Date:                            Tue, 30 Aug 2022   AIC
17083.339
Time:                                    21:46:34   BIC
17122.759
Sample:                                         0   HQIC
17098.307
                                           - 1032
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept    113.3804     32.606      3.477      0.001      49.473     177.288
ar.L1         -0.3390      0.027    -12.425      0.000      -0.393      -0.286
ar.L2          0.5805      0.027     21.424      0.000       0.527       0.634
ma.L1          0.9808      0.018     53.639      0.000       0.945       1.017
ar.S.L12      -0.1357      0.043     -3.133      0.002      -0.221      -0.051
ar.S.L24      -0.1653      0.038     -4.400      0.000      -0.239      -0.092
ma.S.L12      -0.6544      0.039    -16.846      0.000      -0.731      -0.578
sigma2      1.241e+06   3.18e+04     39.026      0.000    1.18e+06     1.3e+06
==============================================================================
===
Ljung-Box (L1) (Q):                   1.36   Jarque-Bera (JB):
4262.89
Prob(Q):                              0.24   Prob(JB):
0.00
Heteroskedasticity (H):               1.00   Skew:
0.94
Prob(H) (two-sided):                  0.98   Kurtosis:
12.84
==========================================================================================
===

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-
step).
```

```
"""
```

[ ]:
```
figure(figsize=(15,10))
# Forecast
n_periods = 48
fitted, confint = sxmodel.predict(n_periods=n_periods, exogenous = np.
  tile(seasonal_index.seasonal, 2).reshape, return_conf_int=True)
index_fcs = pd.date_range(seasonal_index.index[-1], periods = n_periods,␣
  freq='MS')
format['DATE'] = database.index
database.set_index(['DATE'], inplace=True)
# make series for plotting purpose
fitted_series = pd.Series(fitted.values, index=index_fcs)
lower_series = pd.Series(confint[:, 0], index=index_fcs)
upper_series = pd.Series(confint[:, 1], index=index_fcs)
# Plot
plt.plot(format['VALUE'])
plt.plot(fitted_series, color='darkgreen')
plt.plot(upper_series, color='darkred')
plt.plot(lower_series, color='darkblue')
plt.fill_between(lower_series.index, lower_series, upper_series, color='k',␣
  alpha=.15)
plt.title("SARIMAX Forecast")
plt.show()
```

```
/usr/local/lib/python3.7/dist-packages/statsmodels/tsa/base/tsa_model.py:843:
ValueWarning: No supported index is available. Prediction results will be given
with an integer index beginning at `start`.
  data=self.data,
```


SARIMAX Forecast
15000
12500
10000
7500
5000
2500
0
−2500
1930
1940
1950
1960
1970
1980
1990
2000
2010
2020